

Paul Hondola

Timișoara, Romania | +40745166873 | paulhondola@gmail.com | linkedin.com/in/paulhondola | github.com/paulhondola

EDUCATION

Bachelor of Science in Computer Engineering

Oct. 2023 – Jul. 2027 (expected)

Universitatea Politehnica Timișoara | GPA: 9.72/10 | Available Jul. – Sep. 2027

- Relevant Coursework: Data Structures and Algorithms, Parallel and Distributed Algorithms, Operating Systems, Computer Architecture, Formal Languages and Compilation Techniques, Software Architecture, Networking, Databases.

EXPERIENCE

Software Engineer Intern

Mar. 2026 – Present

Nokia | Python, pandas, MariaDB, Jenkins, GitLab, Power BI

- Halved runtime of a high-volume financial data pipeline running 8×/day (2 h → 1 h, ~8 compute-hours saved daily) by replacing sequential row-by-row business logic with vectorized pandas operations; verified equivalence with a prod-vs-dev table diff that exposed silently dropped entries in the legacy implementation.
- Restored 3 product-management data pipelines broken by an upstream access change with a minimal hotfix (HTML → XML export parsing), then proposed and executed a ~90-file rewrite that eliminated duplicated logic through a shared module layer and a uniform index/parser/pipeline structure per data source.

Full Stack Developer Intern

Jul. 2025 – Oct. 2025

Hibyte | TypeScript, Angular, NestJS, Payload CMS, Supabase

- Built the role-based authentication layer (Supabase JWTs validated by NestJS guards across admin, staff and customer roles) and backend routes for a game-center management proof of concept in a 5-person team.
- Configured 7 Payload CMS collections for non-technical admins and maintained Supabase migrations through continuous schema changes.

Reverse Engineering and Malware Analysis Trainee

Apr. 2025 – Jun. 2025

Bitdefender × *LigaAC* | C, Python, x86 Assembly, IDA Pro, Frida

- Reversed a file-encryption routine from x86 disassembly in IDA Pro and implemented working decryptors in C for 5 file formats, plus a keygen reproducing a licence algorithm from recovered lookup tables and hash logic.
- Unpacked dynamically loaded Android DEX payloads with Frida hooks on DexClassLoader/DexFile and built Python/Java tools to decrypt locker-app data across 8 reverse-engineering labs on purpose-built binaries.

PROJECTS

Groundly (Bachelor Thesis) | *Python, FastMCP, SQLite, bge-m3, GraphRAG*

Jul. 2026 – Present

- Built an MCP server (tested with Claude Code and Codex) that indexes course materials into portable single-file knowledge bases, serving hybrid retrieval (bge-m3 dense + learned sparse + BM25, reciprocal rank fusion, cross-encoder reranking) with zero API keys.
- Evaluated 3 GraphRAG retrieval strategies against a reranked hybrid baseline on 124 gold questions across 2 course corpora; after GraphRAG failed to outperform it, removed query-time GraphRAG and the external-model answer pipeline for an MCP-first, zero-key architecture.
- Cut peak indexing memory from >16 GB to a flat 5.05 GB independent of document size by profiling with memray and streaming fp16 embeddings in bounded batches into SQLite.

gemm-bench | *Rust, Rayon, Metal (MPS), Accelerate (AMX), DuckDB, Svelte*

Sep. 2026 – Present

- Benchmarked 10 matrix-multiplication kernels on Apple M1 Pro over 5 precisions and sizes up to 4096: 7 Rust CPU kernels (loop interchange, cache tiling, Rayon work-stealing, static thread pools), Metal (MPS) on the GPU and the AMX coprocessor via Accelerate.
- Raised f32 throughput at $n = 1024$ from 2.1 GFLOPS (naive) to 197 GFLOPS on 10 threads (94×) and 1.75 TFLOPS on AMX, with the GPU peaking at 3.7 TFLOPS; an f64 reference showed MPS f16 ~500× more accurate than CPU f16.
- Backed by 148 tests, GitHub Actions CI and a DuckDB/Parquet pipeline feeding a live DuckDB-WASM dashboard.

Vibecodium | 1st Place, iTEC 2026 | *TypeScript, React, Bun, Supabase, Yjs, Docker*

Mar. 2026 – Jun. 2026

- Co-led a 5-person team building a real-time collaborative IDE with Yjs CRDT multi-user editing, owning the core workspace UI (Monaco editor, file explorer, Git panel, terminal) and the project, session and user REST APIs.
- Built a Docker code-execution sandbox for Python, Node.js, C++ and Rust with per-container 2 GB memory caps, no network access and a 3 s execution timeout.

AtomC Compiler (Compilers coursework) | *C23, Meson*

Mar. 2026 – May. 2026

- Implemented an end-to-end compiler for a C subset in ~3.4K lines of C23 – lexer, recursive-descent parser, scoped symbol tables with struct alignment, type checking and code generation for a 34-opcode stack VM – verified by 8 test suites behind a pre-commit gate with clang-tidy, ASan/UBSan and leak detection.

Distributed Image Processing (Parallel Algorithms coursework) | *C, MPI, OpenMP*

Dec. 2025 – Feb. 2026

- Parallelized 2D convolution on images up to 16800×12600 px across 5 execution models (serial, OpenMP, MPI message passing, shared-file I/O, master-worker task pool), reaching a median 6.8× speedup with 8 MPI processes.

TECHNICAL SKILLS

Languages: Python, C, C++, Rust, Java, TypeScript, SQL, Bash

Parallel and HPC: MPI, OpenMP, Rayon, POSIX Threads, SIMD/NEON, Metal Performance Shaders, Cache Optimization

Systems and Tools: Linux, Docker, Jenkins, GitHub Actions, GitLab CI, Meson, Cargo, Maven, IDA Pro, Frida

Data: pandas, NumPy, DuckDB, Parquet, Power BI, PostgreSQL, MariaDB, SQLite

Web: React, Angular, NestJS, Hono, Svelte, Supabase

Spoken Languages: English (C1/Advanced), Romanian (Native)